



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 0

Integrante(s): 120002850 323064039

323071378 323214636

426059576

Semestre: 2027-1

Fecha de entrega: 21-Agosto-2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	1
2. Desarrollo	1
2.1. Entrenamiento y configuración del LLM (Modelo de Lenguaje de Gran Tamaño) . . .	1
2.2. Comandos UNIX	7
2.3. Introducción a Git	8
2.3.1. ¿Qué es Git?	8
2.3.2. Concepto de repositorio	8
2.3.3. Creación y configuración del repositorio	8
2.4. Introducción a Java	8
2.4.1. Análisis del código	9
2.4.2. Compilación y ejecución en terminal	9
3. Conclusión	11
Referencias	12

1. Introducción

En el panorama tecnológico actual, la programación ya no puede entenderse como una actividad aislada, sino como parte de un entorno de trabajo que integra lenguajes de programación, sistemas operativos, herramientas de control de versiones y tecnologías de apoyo al aprendizaje. Para un estudiante de ingeniería, uno de los principales retos consiste en relacionar estos elementos de manera coherente: aprender la sintaxis de un lenguaje como Java, desenvolverse en un entorno basado en UNIX, gestionar el código mediante Git y comprender el uso responsable de los modelos de lenguaje de gran tamaño (LLM, por sus siglas en inglés).

La realización de esta práctica responde a la necesidad de adquirir, desde el inicio de la formación académica, hábitos y herramientas cercanos a un flujo de trabajo profesional. El uso de comandos UNIX permite interactuar de manera directa y eficiente con el sistema; Git facilita el control de versiones y el seguimiento de los cambios realizados en un proyecto; y la configuración adecuada de un LLM permite aprovechar este tipo de herramientas como apoyo académico de forma consciente y estructurada. Finalmente, Java constituye una introducción al paradigma de Programación Orientada a Objetos, que será fundamental para el desarrollo de la asignatura.

Para abordar estos temas, se establecieron los siguientes objetivos de aprendizaje:

- Configurar un *prompt* especializado que establezca el rol, el contexto y los límites de respuesta de un modelo de lenguaje de gran tamaño (LLM).
- Investigar el concepto y funcionamiento de Git, así como crear un repositorio para la gestión de proyectos posteriores.
- Recopilar y describir algunos de los principales comandos utilizados en una terminal UNIX, explicando brevemente su utilidad para la navegación y el manejo de archivos.
- Realizar una introducción práctica al lenguaje Java mediante la implementación de un programa “Hola Mundo”, estableciendo una primera base para el trabajo del semestre.

2. Desarrollo

2.1. Entrenamiento y configuración del LLM (Modelo de Lenguaje de Gran Tamaño)

En esta etapa de la práctica se configuró el comportamiento del modelo de lenguaje mediante la elaboración de un *prompt*. El propósito fue establecer un conjunto de criterios que orientara sus respuestas y lo convirtiera en una herramienta de apoyo acorde con nuestras necesidades académicas. La formulación clara de instrucciones y contexto es un aspecto central del uso efectivo de este tipo de sistemas [1].

Aunque cada integrante del equipo personalizó su interacción con el modelo, se acordaron varios puntos fundamentales para mantener la utilidad y calidad académica de las respuestas:

- **Especificación del rol del LLM:** Se definió al modelo como un “Asistente Personal de Aprendizaje especializado en Ingeniería en Computación”, con el fin de orientar sus respuestas hacia un contexto técnico y académico.

- **Contexto del bagaje académico:** Se indicó que somos estudiantes de tercer semestre. Esto permite que el modelo ajuste la profundidad de sus explicaciones y distinga entre conocimientos previos y conceptos que todavía requieren desarrollo.
- **Forma de explicación:** Se solicitó una estructura progresiva: comenzar con la intuición, continuar con la explicación conceptual y finalizar con una aplicación práctica, evitando introducir tecnicismos innecesarios sin explicación previa.
- **Aprendizaje activo:** Se estableció que, cuando resulte conveniente, el LLM plantee preguntas o pasos intermedios que ayuden a razonar la solución en lugar de limitarse a proporcionar una respuesta final.
- **Manejo de errores:** Se indicó que, además de corregir un resultado incorrecto, el modelo debe explicar el origen del error y señalar cómo evitarlo en situaciones posteriores.
- **Uso de ejemplos:** Los ejemplos deben aumentar gradualmente en dificultad, comenzando con casos sencillos y avanzando hacia problemas propios del nivel universitario.
- **Manejo de ambigüedad:** Ante una instrucción insuficiente o ambigua, se pidió al modelo evitar suposiciones injustificadas y solicitar la información necesaria para responder con mayor precisión.
- **Formato de las respuestas:** Se solicitó utilizar, cuando sea pertinente, recursos como tablas y bloques de código para mejorar la legibilidad y organización del material de estudio.

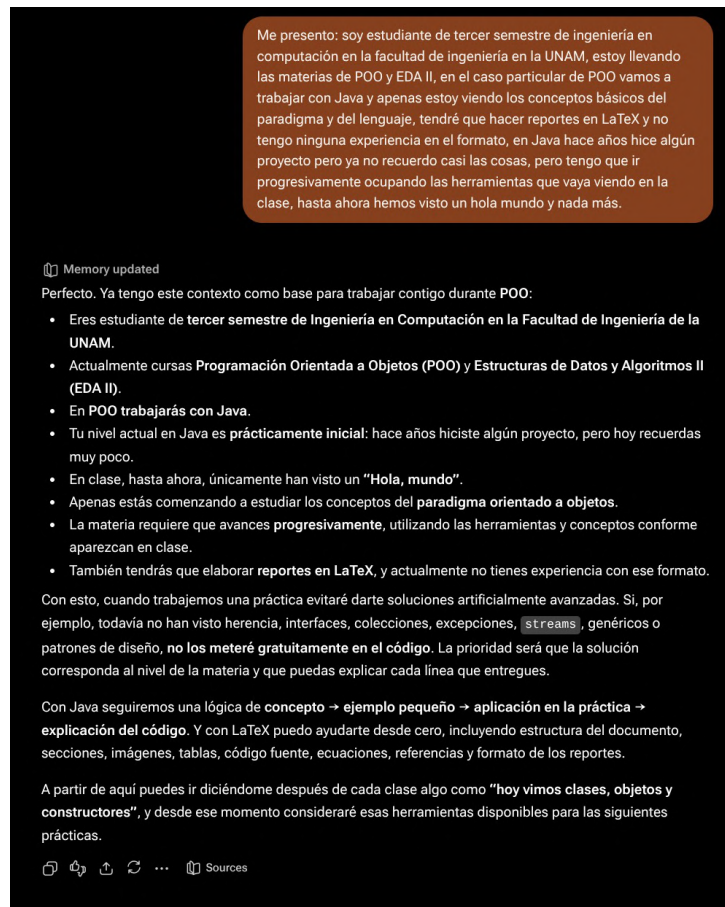


Figura 1: Captura de entrenamiento de Hanniel.

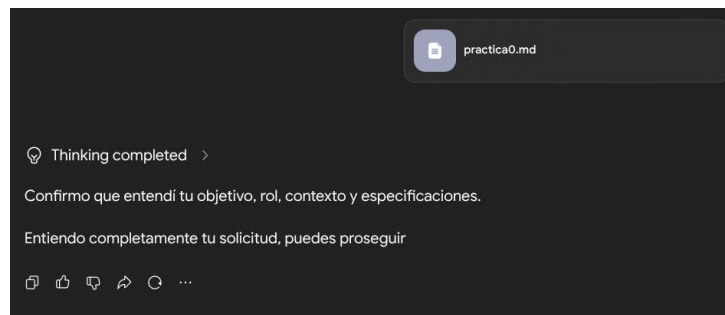


Figura 2: Captura de entrenamiento de Joseph.



Figura 3: Captura de entrenamiento de Alan.

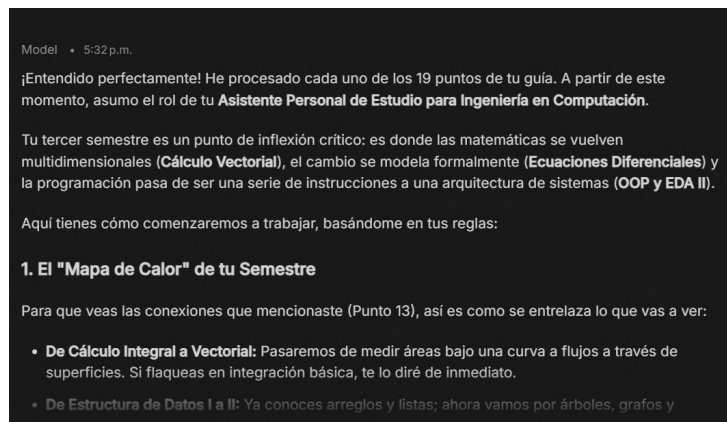


Figura 4: Captura de entrenamiento de Zoe.

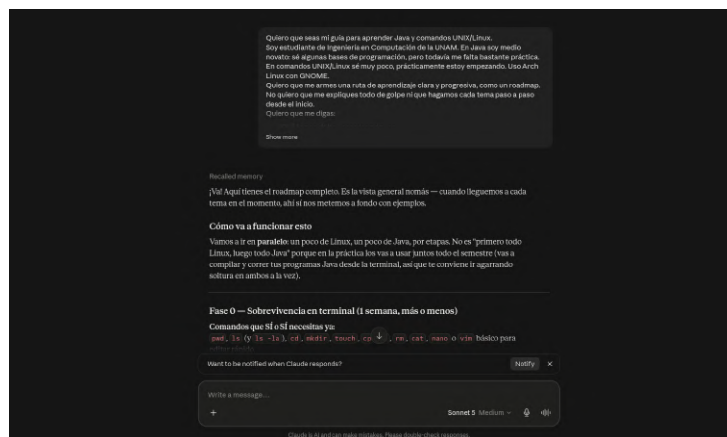


Figura 5: Captura de entrenamiento de Luis.

Dame una tabla corta de Git y Unix

< 2/2 >

Thinking completed >

Git	Unix
<code>git init</code> → crea repositorio local	<code>ls</code> → lista archivos
<code>git clone</code> → descarga repositorio	<code>cd</code> → cambia directorio
<code>git add</code> → prepara cambios	<code>pwd</code> → muestra ruta actual
<code>git commit -m</code> → guarda cambios	<code>cp / mv / rm</code> → copia, mueve, borra
<code>git push / pull</code> → sube/baja cambios	<code>grep</code> → busca texto
<code>git branch / switch</code> → ramas	<code>chmod</code> → permisos
<code>git merge</code> → une ramas	<code>find</code> → busca archivos
<code>git log / status</code> → historial/estado	<code>ps / systemctl</code> → procesos/servicios

📄 🌟 🔍 ↻ ⋮

Figura 6: Ejemplo de respuesta obtenida por Joseph.

Dame una tabla corta de los comandos más utilizados en git y en unix

Claro.

Git

Comando	Función
<code>git status</code>	Ver cambios pendientes
<code>git add .</code>	Preparar todos los cambios
<code>git commit -m "mensaje"</code>	Guardar cambios en Git
<code>git push</code>	Subir cambios a GitHub
<code>git pull</code>	Descargar cambios del repositorio remoto
<code>git clone URL</code>	Copiar un repositorio existente
<code>git init</code>	Crear un repositorio Git
<code>git log</code>	Ver historial de commits

UNIX / Linux

Comando	Función
<code>pwd</code>	Mostrar la carpeta actual
<code>ls</code>	Ver archivos y carpetas
<code>cd carpeta</code>	Entrar a una carpeta
<code>cd ..</code>	Subir una carpeta
<code>mkdir nombre</code>	Crear una carpeta
<code>touch archivo</code>	Crear un archivo vacío
<code>cp</code>	Copiar archivos
<code>mv</code>	Mover o renombrar
<code>rm</code>	Borrar archivos
<code>cat archivo</code>	Mostrar contenido de un archivo
<code>clear</code>	Limpiar la terminal
<code>man comando</code>	Ver el manual de un comando

Figura 7: Ejemplo de respuesta obtenida por Hanniel.

Información relevante según pregunta:

UNIX/Linux — Incluido del día a día

Comando	Qué hace
<code>pwd</code>	Indica en qué carpeta estás parado
<code>ls -la</code>	Muestra archivos detallados: nombre, permisos, tamaño, fecha
<code>cd carpeta</code>	Se mueve a una carpeta
<code>mkdir nombre</code>	Creo una carpeta
<code>touch archivo</code>	Creo un archivo vacío
<code>cp origen destino</code>	Copia
<code>mv origen destino</code>	Mueve o renombra
<code>rm archivo</code>	Borra uno o más archivos
<code>cat archivo</code>	Muestra el contenido completo
<code>git init</code>	Inicia un nuevo repositorio
<code>git add .</code>	Agrega todos los cambios al commit

Git — Se utiliza para trabajar en un proyecto

Comando	Qué hace
<code>git init</code>	Inicia un repositorio en la carpeta actual
<code>git clone URL</code>	Descarga un repositorio
<code>git status</code>	Muestra qué cambios hay en el repositorio
<code>git add archivo</code>	Agrega un archivo al repositorio
<code>git commit -m "mensaje"</code>	Guarda los cambios en el repositorio
<code>git push</code>	Envía los cambios al repositorio remoto
<code>git pull</code>	Descarga los cambios del repositorio remoto
<code>git log --oneline</code>	Muestra el historial de commits
<code>git diff</code>	Muestra los cambios no guardados
<code>git checkout main</code>	Cambia de rama

Con comandos de la terminal puedes crear un repositorio Git y trabajar en él.

¡Gracias!

Figura 8: Ejemplo de respuesta obtenida por Luis.

2.2. Comandos UNIX

Los comandos UNIX son instrucciones que permiten interactuar con el sistema operativo mediante una terminal. Con ellos es posible navegar entre directorios, administrar archivos, ejecutar programas, realizar búsquedas y modificar permisos, entre otras tareas. Su dominio resulta especialmente útil en el desarrollo de software, ya que permite trabajar de forma rápida, precisa y, en muchos casos, automatizable [3].

Cuadro 1: Comandos UNIX de uso frecuente.

Comando	Uso
<code>ls</code>	Lista los archivos y directorios contenidos en la ubicación actual.
<code>cd</code>	Permite cambiar de directorio y navegar por el sistema de archivos.
<code>rm</code>	Elimina archivos; con las opciones adecuadas también puede eliminar directorios.
<code>touch</code>	Crea un archivo vacío o actualiza la fecha de modificación de uno existente.
<code>mkdir</code>	Crea un nuevo directorio.
<code>pwd</code>	Muestra la ruta completa del directorio de trabajo actual.
<code>mv</code>	Mueve archivos o directorios y también permite renombrarlos.
<code>find</code>	Busca archivos o directorios de acuerdo con distintos criterios.
<code>chmod</code>	Modifica los permisos de lectura, escritura y ejecución.
<code>cat</code>	Muestra en la terminal el contenido de uno o varios archivos de texto.
<code>grep</code>	Busca patrones de texto dentro de archivos o de la salida de otros comandos.

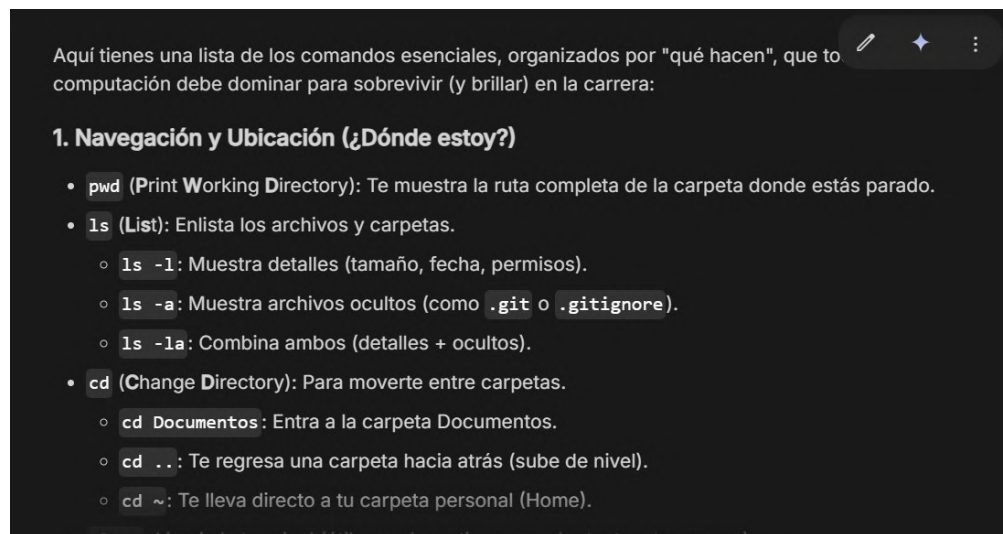


Figura 9: Resultado de la consulta sobre comandos UNIX realizada por Zoe.

2.3. Introducción a Git

2.3.1. ¿Qué es Git?

Git es un sistema distribuido de control de versiones diseñado para registrar los cambios realizados en los archivos de un proyecto a lo largo del tiempo. Su uso permite conservar un historial de modificaciones, recuperar estados anteriores del proyecto y facilitar el trabajo colaborativo entre varios desarrolladores [2].

2.3.2. Concepto de repositorio

Un repositorio es la estructura en la que Git almacena los archivos de un proyecto junto con la información necesaria para mantener su historial de versiones. En un repositorio local, Git crea un directorio oculto llamado `.git`, donde conserva los metadatos, referencias y objetos que permiten reconstruir los distintos estados del proyecto.

2.3.3. Creación y configuración del repositorio

A continuación, se presentan los pasos básicos utilizados para crear y configurar un repositorio local desde la terminal:

1. **Inicialización:** Primero se accede al directorio raíz del proyecto y se ejecuta el siguiente comando:

```
git init
```

Este comando inicializa el directorio actual como un repositorio de Git y crea la estructura interna necesaria para comenzar a registrar cambios.

2. **Configuración de identidad:** Posteriormente se configuran el nombre y el correo electrónico que Git asociará con los commits realizados por el usuario:

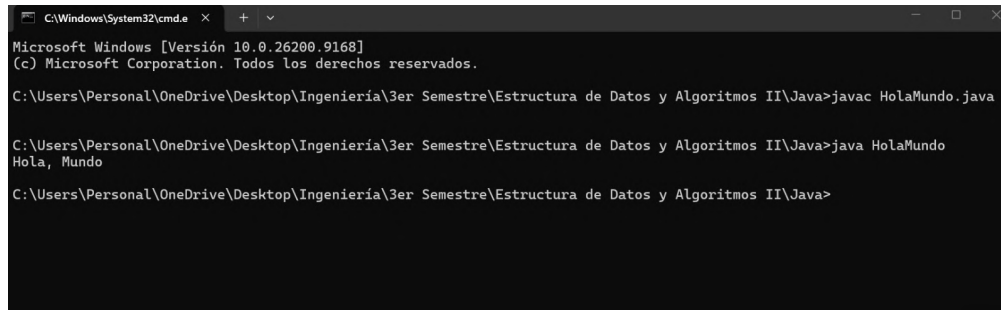
```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@correo.com"
```

La opción `-global` aplica esta configuración a los repositorios utilizados por el mismo usuario en el equipo.

2.4. Introducción a Java

Java es un lenguaje de programación orientado a objetos y de propósito general. En un programa básico, el código se organiza dentro de clases y la ejecución comienza en el método `main`. El siguiente ejemplo corresponde al programa “Hola Mundo” implementado durante la práctica [4]:

```
public class HolaMundo {  
    public static void main(String[] args) {  
        System.out.println("Hola, Mundo");  
    }  
}
```



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.26200.9168]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Personal\OneDrive\Desktop\Ingeniería\3er Semestre\Estructura de Datos y Algoritmos II\Java>javac HolaMundo.java

C:\Users\Personal\OneDrive\Desktop\Ingeniería\3er Semestre\Estructura de Datos y Algoritmos II\Java>java HolaMundo
Hola, Mundo

C:\Users\Personal\OneDrive\Desktop\Ingeniería\3er Semestre\Estructura de Datos y Algoritmos II\Java>
```

Figura 10: Ejecución del programa “Hola Mundo” en Java.

2.4.1. Análisis del código

Para comprender la estructura de este programa inicial, es útil identificar la función de cada uno de sus componentes:

- **public class HolaMundo:** Define una clase pública llamada `HolaMundo`. Cuando una clase pública se declara de esta forma, el archivo debe llamarse `HolaMundo.java`; de lo contrario, el compilador generará un error.
- **public static void main(String[] args):** Declara el método principal del programa y constituye el punto de entrada de la aplicación.
 - *public*: indica que el método es accesible desde fuera de la clase.
 - *static*: permite invocar el método sin crear previamente una instancia de la clase.
 - *void*: indica que el método no devuelve un valor.
 - *String[] args*: representa un arreglo de cadenas que puede recibir argumentos proporcionados al ejecutar el programa desde la terminal.
- **System.out.println(...):** Es la instrucción utilizada para mostrar información en la salida estándar. En este caso, imprime el mensaje `Hola, Mundo` y añade un salto de línea al final.

2.4.2. Compilación y ejecución en terminal

Java emplea un proceso de compilación y ejecución en dos etapas. De manera simplificada, el compilador transforma el código fuente en *bytecode*, y posteriormente la Máquina Virtual de Java (JVM) ejecuta ese código intermedio.

1. **Generación de *bytecode*:** El archivo fuente se compila desde la terminal mediante:

```
javac HolaMundo.java
```

Como resultado se genera el archivo `HolaMundo.class`, que contiene el *bytecode* correspondiente al programa.

2. **Ejecución mediante la JVM:** Una vez compilado, el programa se ejecuta con:

```
java HolaMundo
```

La JVM carga y ejecuta el *bytecode* contenido en el archivo de clase.

Este mecanismo contribuye a la portabilidad de Java, ya que un mismo *bytecode* puede ejecutarse en distintos sistemas que dispongan de una implementación compatible de la JVM.

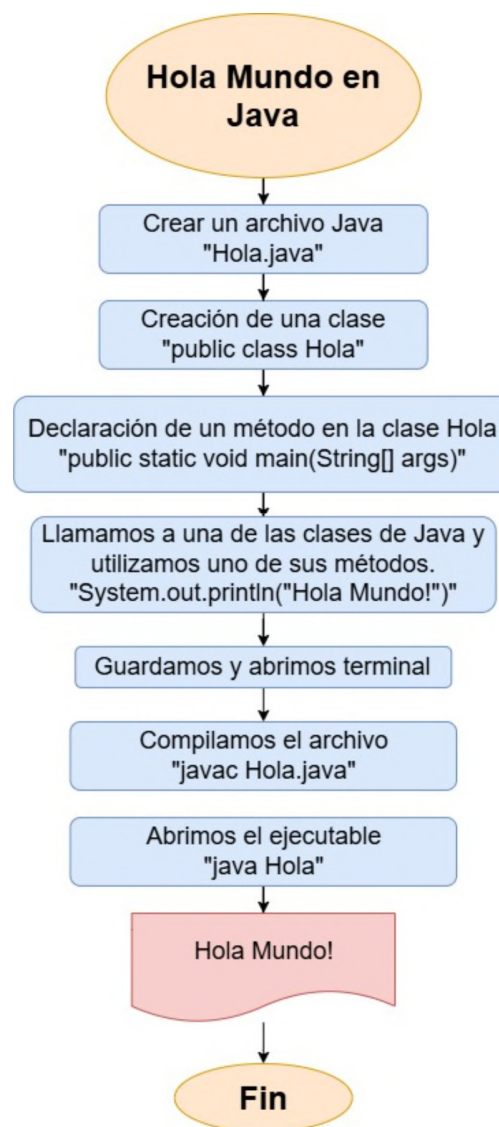


Figura 11: Esquema general del proceso de compilación y ejecución en Java.

3. Conclusión

La realización de esta práctica permitió integrar conocimientos básicos sobre varias herramientas que formarán parte del entorno de trabajo durante el curso. En primer lugar, se comprendió la importancia de proporcionar contexto e instrucciones claras a un modelo de lenguaje para utilizarlo como apoyo académico. Asimismo, se revisaron los fundamentos de Git y la utilidad de los repositorios para conservar el historial de un proyecto, además de algunos comandos UNIX esenciales para desenvolverse desde la terminal.

Finalmente, la implementación y ejecución del programa “Hola Mundo” permitió tener un primer acercamiento práctico a Java, identificar la estructura mínima de un programa y reconocer el proceso de compilación y ejecución mediante la JVM. En conjunto, las actividades realizadas cumplieron los objetivos planteados y establecieron una base común para desarrollar ejercicios y proyectos posteriores con mayor organización y familiaridad con las herramientas empleadas.

Referencias

- [1] Giray, L. (2023). Prompt Engineering with ChatGPT: A Guide for Academic Writers. *Annals of Biomedical Engineering*, 51(12), 2629–2633.
- [2] Astigarraga, J. y Cruz-Alonso, V. (2022). ¡Se puede entender cómo funcionan Git y GitHub! *Ecosistemas*, 31(1), 2332.
- [3] Garcés Guevara, S. (1995). *UNIX: Una puerta abierta a los sistemas abiertos*.
- [4] Deitel, H. M. y Deitel, P. J. (2003). *Cómo programar en Java*. Pearson Educación.